Count each character in a String:

```
static void countOfEachCharacterInString(){
    String input  = "Hello World";
      var result = Arrays.stream(input.split(""))
                          .filter(s -> !s.isBlank())
                          .collect(Collectors.groupingBy(Function.identity(),
                                  Collectors.counting()));

      result.forEach((character, count) ->
               System.out.println(character + " : " + count));

}
```

Count each word in a String:

```
static void countOfEachWordInString(){
    String input  = "Hello World Durga Naresh Hello World";
    var result = Arrays.stream(input.split(" "))
            .filter(s -> !s.isBlank())
            .collect(Collectors.groupingBy(Function.identity(),
                    Collectors.counting()));

    result.forEach((character, count) ->
            System.out.println(character + " : " + count));

}
```

Filter even numbers:

```
static void filterEvenNumbers() {
    List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
    List<Integer> evenNumbers = numbers.stream()
            .filter(n -> n % 2 == 0)
            .toList();
    System.out.println("Even numbers: " + evenNumbers);
}
```

Count strings longer than 5:

```
static void countStringsLongerThanFive() {
    List<String> words = Arrays.asList("hello", "world", "stream",
           "java", "programming");
    long count = words.stream().filter(w -> w.length() > 5).count();
    System.out.println("Strings with length > 5: " + count);
}
```

Map to uppercase:

```
static void mapToUppercase() {
    List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
    List<String> upperNames = names.stream()
            .map(String::toUpperCase)
            .toList();
    System.out.println("Uppercase names: " + upperNames);
}
```

Sum numbers > 10:

```
static void sumNumbersGreaterThanTen() {
    List<Integer> numbers = Arrays.asList(5, 15, 20, 8, 12);
    int sum = numbers.stream().filter(n -> n > 10)
            .mapToInt(Integer::intValue).sum();
    System.out.println("Sum of numbers > 10: " + sum);
}
```

Find first divisible by 7:

```
static void findFirstDivisibleBySeven() {
    List<Integer> numbers = Arrays.asList(5, 14, 21, 28, 3);
    numbers.stream()
            .filter(n -> n % 7 == 0)
            .findFirst()
            .ifPresent(n ->
                  System.out.println("First divisible by 7: " + n));
}
```

List to map word -> length:

```
static void listToMapWordLength() {
    List<String> words = Arrays.asList("apple", "banana", "cherry");
    Map<String, Integer> wordLengthMap = words.stream()
            .collect(Collectors.toMap(
                    w -> w,
                    String::length
            ));
    System.out.println("Word length map: " + wordLengthMap);
}
```

Remove duplicates:

```
static void removeDuplicates() {
    List<Integer> numbers = Arrays.asList(1,2,2,3,3,4,5);
    List<Integer> uniqueNumbers = numbers.stream().distinct().toList();
    System.out.println("Unique numbers: " + uniqueNumbers);
}
```

Group by first letter:

```
static void groupByFirstLetter() {
    List<String> names = Arrays.asList("Alice", "Adam", "Bob",
            "Charlie", "Catherine");
    Map<Character, List<String>> grouped = names.stream()
            .collect(Collectors.groupingBy(n -> n.charAt(0)));
    System.out.println("Grouped by first letter: " + grouped);
}
```

Partition even and odd:

```
static void partitionEvenOdd() {
    List<Integer> numbers = Arrays.asList(1,2,3,4,5,6);
    Map<Boolean, List<Integer>> partitioned = numbers.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0));
    System.out.println(" Even numbers: " + partitioned.get(true));
    System.out.println(" Odd numbers: " + partitioned.get(false));
}
```

Flatten list of lists:

```
static void flattenListOfLists() {
    List<List<Integer>> listOfLists = Arrays.asList(
            Arrays.asList(1,2,3),
            Arrays.asList(4,5,6),
            Arrays.asList(7,8)
    );
    List<Integer> flatList = listOfLists.stream()
            .flatMap(List::stream)
            .toList();
    System.out.println("Flattened list: " + flatList);
}
```

Print First Element:

```
static void example16_PrintFirstElement() {
    List<Integer> nums = Arrays.asList(5, 6, 7);
    nums.stream().findFirst().ifPresent(n -> System.out.println("First element: " + n));
}
```

Optional handling:

```
static void optionalFindStringStartingWithA() {
    List<String> names = Arrays.asList("Bob", "Charlie", "Alice");
    names.stream()
            .filter(n -> n.startsWith("A"))
            .findAny()
            .map(String::toUpperCase)
            .ifPresentOrElse(
                    s -> System.out.println("Found string starting with A: " + s),
                    () -> System.out.println("No string starts with A")
            );
}
```

Sort strings by length descending:

```
static void sortStringsByLengthDesc() {
    List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");
    List<String> sorted = names.stream()
            .sorted((a, b) -> Integer.compare(b.length(), a.length()))
            .toList();
    System.out.println("Sorted by length descending: " + sorted);
}
```

Employee with max salary:

```
static void maxSalaryEmployee() {
    List<Employee> employees = Arrays.asList(
            new Employee("Alice", 5000),
            new Employee("Bob", 7000),
            new Employee("Charlie", 6000)
    );
    employees.stream()
            .max(Comparator.comparingDouble(e -> e.salary))
            .ifPresent(e -> System.out.println("Employee with max salary: " + e));
}
```

Sum Of All Numbers:

```
static void example17_SumAllNumbers() {
    int sum = Stream.of(1,2,3).mapToInt(Integer::intValue).sum();
    System.out.println("Sum all: " + sum);
}
```

Collect To Set:

```
static void example18_CollectToSet() {
    Set<String> set = new HashSet<>(Arrays.asList("a", "a", "b"));
    System.out.println("To set: " + set);
}
```

Distinct Numbers:

```
static void example19_DistinctNumbers() {
    List<Integer> unique = Stream.of(1,1,2).distinct().toList();
    System.out.println("Distinct numbers: " + unique);
}
```

Sort Ascending:

```
static void example20_SortAscending() {
    List<Integer> sorted = Stream.of(3,1,2).sorted().toList();
    System.out.println("Sorted ascending: " + sorted);
}
```

Sort Descending:

```
static void example21_SortDescending() {
    List<Integer> desc = Stream.of(3,1,2)
            .sorted(Comparator.reverseOrder()).toList();
    System.out.println("Sorted descending: " + desc);
}
```

Filter And Sort:

```
static void example22_FilterAndSort() {
    List<Integer> result = Stream.of(5,2,7,4,8)
```

```
            .filter(n -> n > 4).sorted().toList();
    System.out.println("Filter >4 & sort: " + result);
}
```

Square Each Number:

```
static void example23_SquareEachNumber() {
    List<Integer> sq = Stream.of(1,2,3).map(n -> n*n).toList();
    System.out.println("Squares: " + sq);
}
```

Map To String Length:

```
static void example24_MapToStringLength() {
    List<Integer> lengths = Stream.of("a","bb","ccc")
            .map(String::length).toList();
    System.out.println("String lengths: " + lengths);
}
```

Remove Nulls:

```
static void example25_RemoveNulls() {
    List<String> filtered = Stream.of("a", null, "b").filter(Objects::nonNull)
            .toList();
    System.out.println("Remove nulls: " + filtered);
}
```

Sum Of Evens:

```
static void example26_SumOfEvens() {
    int sum = Stream.of(1,2,3,4).filter(n->n%2==0)
            .mapToInt(Integer::intValue).sum();
    System.out.println("Sum of evens: " + sum);
}
```

Max Value:

```
static void example27_MaxValue() {
    Stream.of(2,5,1).max(Integer::compare)
            .ifPresent(n -> System.out.println("Max: " + n));
}
```

Min Value:

```
static void example27_MinValue() {
    Stream.of(2,5,1).min(Integer::compare)
            .ifPresent(n -> System.out.println("Min: " + n));
}
```

First Matching:

```
static void example28_FirstMatching() {
    Stream.of("a","b","c").filter(s->s.startsWith("b"))
            .findFirst().ifPresent(s ->
                    System.out.println("First matching: " + s));
}
```

Any Match:

```
static void example30_AnyMatch() {
    boolean any = Arrays.asList(1,2).stream().anyMatch(n -> n>2);
    System.out.println("Any >2: " + any);
}
```

All Match:

```
static void example31_AllMatch() {
    boolean all = Arrays.asList(2,4,6).stream().allMatch(n -> n%2==0);
    System.out.println("All even: " + all);
}
```

None Match:

```
static void example32_NoneMatch() {
    boolean none = Stream.of(1,3,5).noneMatch(n -> n%2==0);
    System.out.println("None even: " + none);
}
```

GroupBy FirstLetter:

```
static void example33_GroupByFirstLetter() {
    Map<Character,List<String>> grouped = Stream.of("apple","ant","ball")
            .collect(Collectors.groupingBy(s -> s.charAt(0)));
    System.out.println("Group by first char: " + grouped);
}
```

PartitionEvenOdd:

```
static void example34_PartitionEvenOdd() {
    Map<Boolean,List<Integer>> part = Stream.of(1,2,3,4)
            .collect(Collectors.partitioningBy(n->n%2==0));
    System.out.println("Partition even/odd: " + part);
}
```

FlatMapNestedLists:

```
static void example35_FlatMapNestedLists() {
    List<List<Integer>> list = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
    List<Integer> flat = list.stream().flatMap(Collection::stream).toList();
    System.out.println("Flattened: " + flat);
}
```

ReduceConcatenateStrings:

```
static void example24_ReduceConcatenateStrings() {
    String joined = Stream.of("a","b","c").reduce("", String::concat);
    System.out.println("Concatenated: " + joined);
}
```

ReduceMultiplyIntegers:

```
static void example37_ReduceMultiplyIntegers() {
    int product = Stream.of(2,3,4).reduce(1,(a, b) -> a*b);
    System.out.println("Product: " + product);
}
```

JoinWithComma:

```
static void example38_JoinWithComma() {
    String joined = Stream.of("a","b","c").collect(Collectors.joining(","));
    System.out.println("Joined with comma: " + joined);
}
```

JoinWithComma:

```
static void example38_JoinWithComma() {
    String joined = String.join(",", "a", "b", "c");
    System.out.println("Joined with comma: " + joined);
}
```

Top3Highest:

```
static void example27_Top3Highest() {
    List<Integer> top3 = Stream.of(5,1,9,7)
            .sorted(Comparator.reverseOrder()).limit(3).toList();
    System.out.println("Top 3: " + top3);
}
```

SkipFirst2:

```
static void example39_Top3Highest() {
    List<Integer> top3 = Stream.of(5,1,9,7)
            .sorted(Comparator.reverseOrder()).limit(3).toList();
    System.out.println("Top 3: " + top3);
}
```

PeekDebug:

```
static void example40_PeekDebug() {
    System.out.print("Peek debug: ");
    Stream.of(1,2,3,4).filter(n->n%2==0)
            .peek(System.out::print)
            .toList();
    System.out.println();
}
```

SumOfSquaresGreaterThan10:

```
static void example41_SumOfSquaresGreaterThan10() {
    int sum = Stream.of(3,4,5).map(n->n*n).filter(n->n>10)
            .mapToInt(i->i).sum();
    System.out.println("Sum of squares >10: " + sum);
}
```

ConvertToArray:

```
static void example42_ConvertToArray() {
    String[] arr = Stream.of("a","b","c").toArray(String[]::new);
    System.out.println("Array: " + Arrays.toString(arr));
}
```

CountFrequencies:

```
static void example43_CountFrequencies() {
    Map<String,Long> freq = Stream.of("a","a","b")
            .collect(Collectors.groupingBy(s->s, Collectors.counting()));
    System.out.println("Frequencies: " + freq);
}
```

UniqueSortedWords:

```
static void example44_UniqueSortedWords() {
    List<String> unique = Stream.of("b","a","b").distinct().sorted()
            .toList();
    System.out.println("Unique sorted: " + unique);
}
```

LongestString:

```
static void example45_LongestString() {
    Stream.of("aa","bbb","c")
            .max(Comparator.comparingInt(String::length))
            .ifPresent(s -> System.out.println("Longest: " + s));
}
```

ShortestString:

```
static void example46_ShortestString() {
    Stream.of("aa","bbb","c")
            .min(Comparator.comparingInt(String::length))
            .ifPresent(s -> System.out.println("Shortest: " + s));
}
```

EvenNumbersUsingIntStream:

```
static void example47_EvenNumbersUsingIntStream() {
    System.out.print("Even numbers: ");
    IntStream.range(1,6).filter(n->n%2==0).forEach(n->System.out.print(n+" "));
}
```

GenerateFirstNSquares:

```
static void example48_GenerateFirstNSquares() {
    System.out.print("First 5 squares: ");
    IntStream.iterate(1,n->n+1).map(n->n*n)
            .limit(5).forEach(n->System.out.print(n+" "));
}
```

PrimeCheckUsingStreams:

```
static void example49_PrimeCheckUsingStreams() {
    int number = 10;
    boolean isPrime = IntStream.rangeClosed(2, number-1)
            .noneMatch(i -> number % i == 0);
    System.out.println("Is 10 prime? " + isPrime);
}
```

DistinctCountOfNumbers:

```
static void example50_DistinctCountOfNumbers() {
    long count = Stream.of(1,2,2,3).distinct().count();
    System.out.println("Distinct count: " + count);
}
```

FindPalindromes:

```
static void example51_FindPalindromes() {
    List<String> pals = Stream.of("aba","abc","aa")
            .filter(s->new StringBuilder(s).reverse().toString().equals(s)).toList();
    System.out.println("Palindromes: " + pals);
}
```

ConvertListOfIntegersToString:

```
static void example52_ConvertListOfIntegersToString() {
    String result = Stream.of(1,2,3).map(Object::toString)
            .collect(Collectors.joining("-"));
    System.out.println("Integers to string: " + result);
}
```

FilterByPrefix:

```
static void example53_FilterByPrefix() {
    List<String> filtered = Stream.of("apple","bat","ball")
            .filter(s->s.startsWith("b")).toList();
    System.out.println("Starts with b: " + filtered);
}
```

RemoveNegativeNumbers:

```
static void example54_RemoveNegativeNumbers() {
    List<Integer> pos = Stream.of(-1,2,-3,4).filter(n->n>=0).toList();
    System.out.println("Non-negative: " + pos);
}
```

DoubleTheNumbers:

```
static void example55_DoubleTheNumbers() {
    List<Integer> doubled = Stream.of(1,2,3).map(n->n*2).toList();
    System.out.println("Doubled: " + doubled);
}
```

StringsToCharStream:

```
static void example56_StringsToCharStream() {
    List<Character> chars = "abc".chars().mapToObj(c->(char)c).toList();
    System.out.println("Characters: " + chars);
}
```

ParallelStreamSpeedDemo:

```
static void example57_ParallelStreamSpeedDemo() {
    long start = System.currentTimeMillis();
    int sum = IntStream.range(1,1000000).parallel().sum();
    long end = System.currentTimeMillis();
    System.out.println("Parallel sum: " + sum + " in " + (end-start)+"ms");
}
```

AverageOfNumbers:

```
static void example58_AverageOfNumbers() {
    double avg = Stream.of(10,20,30).mapToInt(i->i).average().orElse(0);
    System.out.println("Average: " + avg);
}
```

CollectToTreeSet:

```
static void example59_CollectToTreeSet() {
    TreeSet<String> treeSet =
Arrays.asList("b","a","c").stream().collect(Collectors.toCollection(TreeSet::new));
    System.out.println("TreeSet: " + treeSet);
}
```

```
static void example59_CollectToTreeSet() {
    TreeSet<String> treeSet = new TreeSet<>(Arrays.asList("b", "a", "c"));
    System.out.println("TreeSet: " + treeSet);
}
```

LongestWord:

```
static void example60_LongestWord() {
    Stream.of("a","aa","aaa").max(Comparator.comparingInt(String::length))
            .ifPresent(s->System.out.println("Longest word: " + s));
}
```

StatisticsSummary:

```
static void example61_StatisticsSummary() {
    IntSummaryStatistics stats = Stream.of(1,2,3,4).mapToInt(i->i)
            .summaryStatistics();
    System.out.println("50. Stats: " + stats);
}
```